

Aula 17 - Views, Stored Procedures e Triggers

Descrição: Compreender **Views**, **Stored Procedures** e **Triggers** é um marco importante na jornada de aprendizado em banco de dados. Esses três recursos transformam o SQL de uma simples linguagem de consulta em uma ferramenta poderosa para encapsular lógica de negócios, garantir segurança e automatizar tarefas diretamente no servidor do banco de dados.

1. Views (Visões)

Uma **View** é essencialmente uma "tabela virtual". Ela não armazena dados físicos por si só; em vez disso, ela armazena uma instrução **SELECT** no banco de dados. Toda vez que você consulta uma View, o banco de dados executa a query subjacente e retorna os resultados como se fossem uma tabela real.

A. Por que usar?

- **Segurança:** Você pode restringir o acesso a dados sensíveis. Em vez de dar acesso à tabela inteira de Funcionários (que pode ter salários e CPFs), você cria uma View que mostra apenas os nomes e departamentos, e dá acesso apenas à View.
- **Simplicidade:** Permite encapsular queries extremamente complexas (com vários JOINS, agregações e filtros) em um único objeto fácil de consultar.
- **Consistência:** Garante que todos os usuários e aplicações usem a mesma lógica para buscar uma informação específica.

B. Exemplo Prático

Imagine que você tem as tabelas **Clientes** e **Pedidos**. Você quer ver frequentemente os clientes que gastaram mais de R\$1.000,00 sem ter que reescrever os JOINS toda vez.

Criando a View:

```
SQL
CREATE VIEW v_Clientes_VIP AS
SELECT
    c.ID_Cliente,
    c.Nome,
    SUM(p.Valor_Total) AS Total_Gasto
FROM Clientes c
JOIN Pedidos p ON c.ID_Cliente = p.ID_Cliente
GROUP BY c.ID_Cliente, c.Nome
HAVING SUM(p.Valor_Total) > 1000.00;
```

Usando a View:

```
SQL
-- Agora você pode consultá-la como se fosse uma tabela comum
SELECT * FROM v_Clientes_VIP WHERE Nome LIKE 'A%';
```

C. Explicando o código

Vamos destrinchar esse código linha por linha para entender exatamente o que o banco de dados está fazendo na criação e na utilização dessa View.

Entendendo a Criação da View

O primeiro bloco de código é responsável por definir e salvar a "tabela virtual" no banco de dados.

- `CREATE VIEW v_Clientes_VIP AS`

Este é o comando que diz ao banco de dados: "Crie um objeto do tipo View e dê a ele o nome de `v_Clientes_VIP`". O prefixo `v_` é uma boa prática de mercado para bater o olho e saber que se trata de uma View e não de uma tabela física. O `AS` indica que tudo o que vier a seguir será a regra dessa View.

- `SELECT c.ID_Cliente, c.Nome, SUM(p.Valor_Total) AS Total_Gasto`

Aqui definimos quais colunas farão parte da View. Estamos pegando o ID e o Nome do cliente. A parte `SUM(p.Valor_Total)` soma os valores de todos os pedidos daquele cliente. O `AS Total_Gasto` apenas renomeia essa coluna calculada para que ela tenha um nome amigável quando consultarmos a View.

- `FROM Clientes c`

Indica a tabela principal de onde estamos puxando os dados. A letra `c` é um *alias* (apelido). Em vez de digitar `Clientes.Nome` o tempo todo, podemos digitar apenas `c.Nome`.

- `JOIN Pedidos p ON c.ID_Cliente = p.ID_Cliente`

Esta linha cruza os dados da tabela de Clientes com a tabela de Pedidos (apelidada de `p`). O `ON` define a regra do cruzamento: o banco de dados vai alinhar as linhas onde o `ID_Cliente` da tabela Clientes for exatamente igual ao `ID_Cliente` da tabela Pedidos.

- `GROUP BY c.ID_Cliente, c.Nome`

Como usamos a função de soma (`SUM`) lá no `SELECT`, precisamos agrupar os dados. O `GROUP BY` diz ao banco: "Some os valores dos pedidos, mas crie um subtotal para cada cliente (baseado no ID e no Nome)". Se um cliente tem 5 pedidos, o banco vai juntar os 5 em uma única linha e apresentar o valor total.

- `HAVING SUM(p.Valor_Total) > 1000.00;`

O `HAVING` funciona como um `WHERE`, mas é exclusivo para dados agrupados e funções matemáticas (como `SUM`, `COUNT`, `AVG`). Aqui, estamos dizendo: "Depois de somar tudo, descarte os clientes cujo total não ultrapasse R\$ 1.000,00". Apenas os que gastaram mais que isso entram na View.

Entendendo o Uso da View

O segundo bloco mostra como a sua vida fica mais fácil após a criação da View.

- `SELECT * FROM v_Clientes_VIP`

Em vez de escrever todo aquele bloco complexo de `JOIN`, `GROUP BY` e `SUM`, você faz um `SELECT` simples e direto. O banco de dados vai nos bastidores e executa a lógica que você salvou.

- `WHERE Nome LIKE 'A%';`

Como a View se comporta como uma tabela normal, você pode aplicar filtros adicionais nela. O comando `LIKE 'A%'` filtra o resultado final para mostrar **apenas os clientes VIP cujo nome comece com a letra A**. O símbolo `%` significa "qualquer texto que venha depois da letra A".

2. Stored Procedures (Procedimentos Armazenados)

Um **Stored Procedure** é um bloco de código SQL (que pode incluir lógica de programação como IF/ELSE, loops e tratamento de erros) que é salvo no banco de dados para ser reutilizado.

A. Por que usar?

- **Reutilização e Manutenção:** A lógica fica no banco de dados. Se a regra de negócio mudar, você altera apenas a Procedure, e todas as aplicações (web, mobile, desktop) que a consomem passam a usar a regra nova instantaneamente.
- **Desempenho:** As Procedures são pré-compiladas pelo banco de dados, o que pode reduzir o tempo de execução. Além disso, reduzem o tráfego de rede, pois você envia apenas a chamada da Procedure, e não centenas de linhas de código SQL.
- **Segurança:** Previne ataques de Injeção de SQL e permite que usuários executem operações complexas sem ter permissões diretas nas tabelas subjacentes (apenas permissão para executar a Procedure).

B. Exemplo Prático

Vamos criar uma Procedure para inserir um novo pedido, que aceita parâmetros de entrada.

Criando a Procedure (Sintaxe estilo MySQL):

```
SQL
DELIMITER //

CREATE PROCEDURE sp_InserirNovoPedido (
  IN p_ID_Cliente INT,
  IN p_Valor DECIMAL(10,2)
)
BEGIN
  -- Lógica para inserir o pedido
  INSERT INTO Pedidos (ID_Cliente, Data_Pedido, Valor_Total)
  VALUES (p_ID_Cliente, CURRENT_DATE, p_Valor);

  -- Retornar uma mensagem de sucesso
  SELECT 'Pedido inserido com sucesso!' AS Mensagem;
END //

DELIMITER ;
```

Executando a Procedure:

```
SQL
CALL sp_InserirNovoPedido(15, 250.50);
```

C. Explicando o código

Vamos destrinchar agora o código da **Stored Procedure**. Como ela envolve lógica de programação (parâmetros, início e fim de blocos), a sintaxe tem alguns detalhes bem interessantes.

Entendendo a Criação da Procedure

- **DELIMITER //**

Por padrão, o banco de dados usa o ponto e vírgula (;) para saber que uma instrução SQL terminou e deve ser executada. Porém, a nossa Procedure terá vários pontos e vírgulas dentro dela. Se deixássemos o padrão, o banco tentaria executar a Procedure pela metade assim que achasse o primeiro ;.

O `DELIMITER //` diz ao banco: "Ei, temporariamente, mude o sinal de término de ; para //".

- `CREATE PROCEDURE sp_InserirNovoPedido (`

Aqui começamos a criar o objeto. `sp_` é o prefixo recomendado para *Stored Procedure*. Abrimos parênteses para definir o que ela vai receber de fora.

- `IN p_ID_Cliente INT, IN p_Valor DECIMAL(10,2)`

Estes são os **parâmetros de entrada** (variáveis que a Procedure precisa para trabalhar):

- `IN`: Indica que é um parâmetro que entra na Procedure.
- `p_ID_Cliente` e `p_Valor`: São os nomes das variáveis. Usar o prefixo `p_` é uma ótima prática para você não confundir o nome do parâmetro com o nome das colunas da tabela.
- `INT` e `DECIMAL(10,2)`: São os tipos de dados que essas variáveis aceitam (um número inteiro e um número decimal com duas casas decimais, respectivamente).

- `) BEGIN`

Fechamos os parâmetros e usamos o `BEGIN` para indicar: "Aqui começa o bloco de código que será executado".

- `INSERT INTO Pedidos (ID_Cliente, Data_Pedido, Valor_Total)`

`VALUES (p_ID_Cliente, CURRENT_DATE, p_Valor);`

Esta é a ação que a Procedure faz. Ela insere uma linha na tabela `Pedidos`. Note que ela joga o `p_ID_Cliente` (que você passou) na coluna `ID_Cliente`, usa uma função do próprio banco (`CURRENT_DATE`) para preencher a data de hoje automaticamente, e joga o `p_Valor` na coluna `Valor_Total`.

- `SELECT 'Pedido inserido com sucesso!' AS Mensagem;`

Uma Procedure pode retornar dados. Aqui, usamos um `SELECT` simples com um texto fixo para que, assim que a inserção termine, a aplicação ou o usuário receba uma confirmação visual na tela de que tudo deu certo.

- `END //`

O `END` fecha o bloco de código da Procedure. Lembra que mudamos o delimitador lá no início? Usamos o `//` aqui para avisar ao banco que a Procedure acabou de ser escrita por completo e já pode ser salva/compilada.

- `DELIMITER ;`

Por fim, essa linha devolve o comportamento do banco ao normal, definindo o ponto e vírgula (;) novamente como o finalizador padrão de comandos.

Entendendo a Execução da Procedure

Depois que ela está salva no banco, executá-la é muito simples:

- `CALL sp_InserirNovoPedido(15, 250.50);`
- O comando `CALL` (chamar) ativa a Procedure. Entre parênteses, passamos os valores na mesma

ordem em que os parâmetros foram criados:

- O número 15 é enviado para o p_ID_Cliente.
- O valor 250.50 é enviado para o p_Valor.

O banco processa tudo silenciosamente e devolve na tela a tabela com a mensagem: *"Pedido inserido com sucesso!"*.

3. Triggers (Gatilhos)

Um **Trigger** é um tipo especial de **Stored Procedure** que não é chamado manualmente. Em vez disso, ele é "disparado" (executado) **automaticamente** pelo banco de dados quando um evento específico ocorre em uma tabela. Esses eventos são as operações de **modificação de dados**: INSERT, UPDATE ou DELETE.

A. Por que usar?

- **Auditoria:** Registrar quem alterou um registro e quando.
- **Integridade de Dados e Regras de Negócio Automáticas:** Por exemplo, deduzir o estoque de um produto automaticamente assim que uma venda é registrada.
- **Sincronização:** Atualizar tabelas relacionadas que não possuem regras de chave estrangeira com cascata.

Atenção: Triggers devem ser usados com cuidado. Como são invisíveis para a aplicação (rodam em background no banco), Triggers em excesso ou mal escritos podem causar problemas de performance e "efeitos colaterais" difíceis de debugar.

B. Como aplicar (Exemplo Prático)

Vamos criar um gatilho que registra automaticamente uma entrada de auditoria toda vez que o salário de um funcionário for atualizado.

Criando a Tabela de Auditoria:

SQL

```
CREATE TABLE Auditoria_Salario (  
    ID_Auditoria INT AUTO_INCREMENT PRIMARY KEY,  
    ID_Funcionario INT,  
    Salario_Antigo DECIMAL(10,2),  
    Salario_Novo DECIMAL(10,2),  
    Data_Alteracao DATETIME  
);
```

Criando o Trigger (Sintaxe estilo MySQL):

SQL

DELIMITER //

```
CREATE TRIGGER trg_AposAtualizarSalario  
AFTER UPDATE ON Funcionarios -- Dispara DEPOIS de um UPDATE na tabela Funcionarios  
FOR EACH ROW -- Executa para cada linha afetada  
BEGIN  
    -- Verifica se o salário realmente mudou  
    IF OLD.Salario <> NEW.Salario THEN
```

```
INSERT INTO Auditoria_Salario (ID_Funcionario, Salario_Antigo, Salario_Novo, Data_Alteracao)
VALUES (NEW.ID_Funcionario, OLD.Salario, NEW.Salario, NOW());
END IF;
END //
```

DELIMITER ;

Nota: **OLD** refere-se aos dados antes da atualização, e **NEW** refere-se aos dados após a atualização.

C. Explicando o código

Para fechar vamos destrinchar como o **Trigger** e a tabela de auditoria funcionam em conjunto. O conceito do Trigger pode parecer "mágico" porque você não o vê executando na aplicação, mas o código revela exatamente como ele age nos bastidores.

1. Entendendo a Tabela de Auditoria

Antes do Trigger funcionar, ele precisa de um lugar para guardar as informações. Por isso criamos a tabela `Auditoria_Salario`.

- `CREATE TABLE Auditoria_Salario (`
Comando padrão para criar uma nova tabela no banco de dados.
- `ID_Auditoria INT AUTO_INCREMENT PRIMARY KEY,`
Esta é a chave principal da tabela de auditoria. O `AUTO_INCREMENT` garante que o próprio banco de dados vai gerar os números sequenciais (1, 2, 3...) toda vez que um novo registro entrar. Você não precisa se preocupar em preencher isso.
- `ID_Funcionario INT,`
Guarda qual funcionário teve o salário alterado.
- `Salario_Antigo DECIMAL(10,2), e Salario_Novo DECIMAL(10,2),`
As duas colunas cruciais da auditoria: vão armazenar o "antes" e o "depois" da mudança.
- `Data_Alteracao DATETIME`
Guarda a data e a hora exatas em que a modificação ocorreu. O fechamento com `);` conclui a criação da tabela.

2. Entendendo a Criação do Trigger

Aqui é onde a automação acontece. O código define a "armadilha" que vai disparar a auditoria.

- `DELIMITER // e (no final) DELIMITER ;`
Assim como na Stored Procedure, usamos isso para trocar o delimitador padrão (;) por //, permitindo que o bloco do Trigger tenha vários pontos e vírgulas dentro dele sem ser interrompido precocemente.
- `CREATE TRIGGER trg_AposAtualizarSalario`
Cria o objeto do tipo Trigger. O prefixo `trg_` é a convenção recomendada, seguido de um nome descritivo.
- `AFTER UPDATE ON Funcionarios`

Esta linha é o "coração" do Trigger. Ela diz exatamente **quando** e **onde** ele deve agir:

- **AFTER**: Dispara *depois* que a alteração for salva. (Poderia ser **BEFORE** para validar dados antes de salvar).
 - **UPDATE**: Só reage ao comando de atualização. Se alguém inserir (**INSERT**) ou deletar (**DELETE**) um funcionário, este Trigger fica quieto.
 - **ON Funcionarios**: É a tabela que está sendo "vigiada".
-
- **FOR EACH ROW**
Instrução fundamental. Ela diz ao banco: *"Se um comando **UPDATE** alterar o salário de 50 funcionários de uma vez, execute este Trigger 50 vezes (uma para cada linha afetada)"*.
-
- **BEGIN**
Inicia a lógica de programação do que o Trigger vai fazer.
-
- **IF OLD.Salario <> NEW.Salario THEN**
Esta é uma proteção excelente. Imagine que alguém faça um **UPDATE** no endereço do funcionário. O Trigger vai disparar (pois houve um **UPDATE** na tabela), mas o salário não mudou.
Aqui, o banco compara o pseudo-registro **OLD** (dados como estavam no banco antes do update) com o **NEW** (dados que o usuário enviou no update). O símbolo **<>** significa "diferente". A lógica é: *"Se o salário antigo for diferente do salário novo, então continue"*.
-
- **INSERT INTO Auditoria_Salario (ID_Funcionario, Salario_Antigo, Salario_Novo, Data_Alteracao)**
Se o **IF** anterior for verdadeiro, o Trigger insere uma nova linha na nossa tabela de auditoria.
-
- **VALUES (NEW.ID_Funcionario, OLD.Salario, NEW.Salario, NOW());**
Aqui extraímos as informações dos pseudo-registros e salvamos:
 - Pegamos o ID do funcionário usando o **NEW.ID_Funcionario**.
 - Gravamos o "antes" usando **OLD.Salario**.
 - Gravamos o "depois" usando **NEW.Salario**.
 - A função **NOW()** pega a data e hora do sistema do servidor naquele exato segundo.
-
- **END IF; e END //**
O **END IF;** fecha a condição do salário. O **END //** fecha todo o bloco do Trigger para que o banco possa compilá-lo.
- Com isso no lugar, se alguém executar no sistema um **UPDATE Funcionarios SET Salario = 5000 WHERE ID = 10;**, o banco faz o update na tabela principal e, uma fração de segundo depois, o Trigger automaticamente registra o histórico completo sem que o usuário sequer perceba.
-

Fechamento: Elevando o Nível do seu SQL

Recapitulando os pontos principais de hoje:

- **Views**: Atuam como "tabelas virtuais" baseadas em consultas salvas. Elas são a escolha certa para simplificar leituras e garantir a segurança, restringindo o acesso apenas aos dados necessários.
- **Stored Procedures**: São blocos de código SQL salvos que podem ser executados manualmente. Elas

brilham na centralização de lógicas de negócio complexas e rotinas, garantindo fácil manutenção e reutilização por diversas aplicações.

- **Triggers:** São executados de forma totalmente automática sempre que ocorre um evento de modificação (como INSERT, UPDATE ou DELETE). São essenciais para processos de auditoria e automação de regras implícitas nos bastidores.

Com essas três ferramentas, você agora tem a capacidade de construir arquiteturas de banco de dados muito mais robustas, eficientes e seguras!

Resumo Comparativo

Característica	View	Stored Procedure	Trigger
O que é?	Consulta SQL salva (Tabela Virtual).	Bloco de código SQL salvo.	Bloco de código ativado por eventos.
Execução	Manual (usando SELECT).	Manual (usando CALL ou EXEC).	Automática (INSERT, UPDATE, DELETE).
Aceita Parâmetros?	Não.	Sim (Entrada e Saída).	Não.
Uso Principal	Simplificar consultas e segurança de leitura.	Lógica de negócios complexa e rotinas.	Auditoria e automação de regras implícitas.